
Chess Engine + NNUE

The Complete Learning Guide

Everything in this project, explained twice — once in plain English, once for the interview. The architecture, the search, the neural network, every efficiency trick, every design choice and the alternative we rejected, the bugs we hunted, and a glossary so no piece of jargon is left undefined.

How to read this. Each topic has two coloured boxes. **IN PLAIN ENGLISH** is the no-jargon version. **UNDER THE HOOD** is the precise, technical version you would say in an interview. Amber **THE DECISION & WHY** boxes explain what we picked, what we rejected, and the reason — the questions interviewers love most.

A note on authorship: you drove every decision in this build — you set the goals, chose between the options laid in front of you, and found the symptoms that led to the real bugs. This guide exists so the concepts behind those decisions are fully yours.

What's inside

- 1 — The 30,000-foot view: what we actually built
- 2 — Chess-engine foundations (bitboards, legal moves, hashing)
- 3 — Why C++ compiled to WebAssembly (and not JavaScript, and not a server)
- 4 — How the AI looks ahead: the search
- 5 — How the AI judges a position: evaluation (HCE vs NNUE)
- 6 — NNUE, properly: what a neural net is and why this one is shaped this way
- 7 — The efficiency tricks (the heart of the project)
- 8 — The opening book
- 9 — Training the neural net
- 10 — The front end (and why it's plain JavaScript)
- 11 — Every design choice in one table (interview gold)
- 12 — War stories: the bugs and what they taught us
- 13 — Mock interview: questions you can now answer
- 14 — Glossary: every piece of jargon, defined

1 The 30,000-foot view: what we actually built

A chess website where you can play a human or an AI. The twist is that the chess rules and the AI are not written in JavaScript — they are written in C++, compiled to WebAssembly, and run inside the browser at near-native speed. There is no server doing the thinking; everything happens on the player's own machine. The AI can judge positions two different ways: with a hand-written rulebook, or with a neural network we trained from scratch.

IN PLAIN ENGLISH

Think of the project as three machines stacked together. The first machine knows the **rules** (what moves are legal). The second machine **looks ahead** ("if I go here, you go there, then I go there...") and picks the best move. The third machine is the second machine's **sense of taste** — it scores how good a position is so the look-ahead has something to aim for. On top of all that sits a **book of openings** so the AI plays known good moves instantly at the start, and a **web page** that draws the board.

UNDER THE HOOD

The layers are: (1) a bitboard move generator and rules engine in C++; (2) an alpha-beta search with iterative deepening, a transposition table, move ordering and several pruning heuristics; (3) an evaluation function behind an abstract Evaluator interface, implemented twice — a hand-crafted eval (HCE) and a quantized, incrementally-updated NNUE. A Polyglot opening book is probed in JavaScript before search. The C++ compiles to a wasm32 module that runs in a Web Worker; a vanilla-JS front end only renders.

The one rule that shapes everything: a single source of truth

The C++ engine is the only thing allowed to decide what is legal. The web page never works out moves on its own — it asks the engine "what are the legal moves from this square?" and only draws those. That means the user interface literally cannot let you make an illegal move, and the exact same legal-move generator feeds both the on-screen dots and the AI's search. No duplicated logic, no chance of the display and the engine disagreeing.

2 Chess-engine foundations

Rules versus AI — two different jobs

A chess program has two separable halves. The **rules** half answers factual questions: is this move legal, is this checkmate, is this a draw. The **AI** half answers an opinion question: of the legal moves, which is best. We built the rules half first and completely, because the AI is worthless if it can consider illegal moves.

Bitboards — the board as 64 bits

IN PLAIN ENGLISH

A chess board has 64 squares. A computer's natural number is 64 bits — 64 on/off switches. So we store "where are the white pawns?" as a single 64-bit number with a 1 on every square that has a white pawn and a 0 everywhere else. Asking questions about the board then becomes plain arithmetic, which a processor does astonishingly fast.

UNDER THE HOOD

Each piece type/colour gets a `uint64_t` (a U64 bitboard). Move generation, attacks and occupancy are bitwise operations — shifts, AND, OR, XOR — plus two hardware-friendly primitives: `popcount` (how many bits are set = how many pieces) and `lsb/pop_lsb` (find/remove the lowest set bit = iterate pieces). Sliding-piece attacks (bishop/rook/queen) use precomputed attack tables built at startup.

THE DECISION & WHY

Bitboards over a "mailbox" array. The obvious alternative is an array of 64 entries, each naming the piece on that square. It is easier to read but far slower: counting or moving pieces means looping over squares. Bitboards turn those loops into single CPU instructions, which matters enormously when the search visits millions of positions per move. We paid a little readability for a lot of speed — the right trade for an engine.

Generating legal moves (and the check filter)

Generating moves happens in two passes. First we generate **pseudo-legal** moves — every move that follows how a piece moves, ignoring whether it leaves your own king in check. Then we **filter**: make the move, ask "is my king attacked?", and throw the move away if so. Castling, en passant and promotion are handled as special cases. What survives the filter is the one and only definition of "legal" used everywhere — the UI highlights, the human's moves, and the AI search all consume the same list.

Zobrist hashing — a fingerprint for a position

IN PLAIN ENGLISH

We often need to ask "have I seen this exact position before?" Comparing whole boards is slow, so instead we boil each position down to a single 64-bit fingerprint. Same position → same fingerprint. It is like a hash of a password, but for a chess position.

UNDER THE HOOD

Zobrist hashing assigns a random 64-bit key to every (piece, square) combination, plus keys for side-to-move, castling rights and en-passant file. The position's key is the XOR of all the keys that apply. Because XOR is its own inverse, the key can be updated incrementally as pieces move. It powers the transposition table and threefold-repetition detection.

3 Why C++ compiled to WebAssembly

What WebAssembly (WASM) even is

IN PLAIN ENGLISH

Browsers natively understand JavaScript, which is convenient but not very fast. WebAssembly is a way to take code written in a fast language (here, C++), compile it to a compact low-level format, and run *that* in the browser at close to the speed of a normal installed program. JavaScript paints the screen; WebAssembly does the heavy thinking.

UNDER THE HOOD

WASM is a portable binary instruction format with a compact stack-machine bytecode. It runs in the same sandbox as JS, is typically within ~1.5x of native speed, and is supported by every modern browser. We compile the engine to the wasm32 target. JS and C++ talk through a tiny C ABI: the C++ exposes plain functions (no name-mangling) and shares data through the WASM linear memory — JS writes bytes into a buffer, calls an exported function, reads bytes back.

THE DECISION & WHY

WASM over plain JavaScript. A chess search visits millions of positions per move. JavaScript's just-in-time compiler is good, but the predictable, allocation-free, integer-heavy inner loops of a chess engine run far faster as compiled C++. We keep JS for what it is best at — the DOM and rendering — and push every hot loop into WASM.

THE DECISION & WHY

WASM in the browser over a backend server. The classic alternative is: browser sends the position to a server, the server runs the engine, sends back a move. We rejected that. Doing it client-side means **no server to run or pay for, no network latency** per move, **it scales for free** (every user brings their own CPU), and the whole thing hosts as static files on GitHub Pages or Cloudflare Pages. The cost is that strength is bounded by the player's device, which is fine for this project.

The Evaluator interface — designing for a brain transplant

Before writing any neural network, the search was built to talk to an abstract Evaluator interface rather than to a specific scoring function. The hand-crafted eval and, later, the NNUE both implement that interface. Swapping the brain the search uses changes nothing about the search code itself.

THE DECISION & WHY

An abstraction layer up front, before we needed it. It would have been quicker to wire the hand-crafted eval directly into the search. By instead hiding evaluation behind an interface, dropping in NNUE later (and letting the user switch between the two at runtime) became a clean, low-risk change. This is the classic "program to an interface" decision, and it paid off exactly as intended.

The toolchain: Zig instead of Emscripten

IN PLAIN ENGLISH

To turn C++ into WebAssembly you need a compiler. The famous one is Emscripten. It was blocked on the network where this was built, so we used the compiler bundled with the Zig toolchain instead. It produces a standard WASM module just the same.

UNDER THE HOOD

We compile with Zig's bundled clang/lld targeting wasm32-freestanding, with `-O3 -msimd128 -fno-exceptions -fno-rtti -nostdlib`. Because it is freestanding (no standard library, no Emscripten glue/embind), the JS↔C++ boundary is a hand-written C ABI over shared memory rather than generated bindings. The output is a normal WASM module, so it is Emscripten-buildable later without touching the engine.

THE DECISION & WHY

Zig's clang over Emscripten. Forced by the environment (Emscripten was network-blocked), but it also kept the toolchain tiny and the output standard. The trade-off: we wrote the JS/C++ marshalling by hand instead of getting it generated. For a small, fixed API surface that was a perfectly acceptable amount of glue.

Shipping the WASM: base64-embedded, not fetched

IN PLAIN ENGLISH

The compiled engine could be a separate file the page downloads. Instead we paste it into a JavaScript file as text. That way the page works even when opened directly, with no separate download to go wrong.

UNDER THE HOOD

The `.wasm` bytes are base64-encoded into `frontend/js/engine-wasm.js` as a string the page and the Web Worker both decode and instantiate. This sidesteps a second network request and any CORS/MIME issues with serving `.wasm`, and it even works from `file://`. The cost is roughly a one-third size increase from base64 — negligible here.

The search runs on a background thread (Web Worker)

IN PLAIN ENGLISH

If the AI did all its thinking on the same thread that draws the page, the page would freeze solid for the whole think. So the search runs on a separate background worker; the board stays smooth and responsive while the AI thinks.

UNDER THE HOOD

The search runs in a Web Worker — a separate JS thread with its own copy of the WASM engine. The main thread sends the position (as a FEN string) and the depth/time budget; the worker runs iterative deepening against a wall-clock deadline and posts back the best move. The main thread never blocks, so rendering and input stay at 60fps while the AI thinks.

4 How the AI looks ahead: the search

"Searching" means looking ahead through possible moves and replies, scoring the resulting positions, and backing those scores up to choose the best move now. Every technique below exists to do that look-ahead either more *correctly* or much more *cheaply*.

Minimax — assume both sides play their best

IN PLAIN ENGLISH

I pick the move that is best for me, assuming you will then reply with the move that is best for you, and so on. I am maximising my outcome while expecting you to minimise it — hence "minimax".

UNDER THE HOOD

Implemented as negamax: from each position we score from the side-to-move's point of view and negate when we recurse, which collapses the min and max cases into one. Leaves are scored by the evaluator; interior nodes take the best of the negated child scores.

Alpha-beta pruning — skip lines that can't matter

IN PLAIN ENGLISH

While searching, if you discover that a move lets the opponent do something so good that you would never allow it, you can stop examining that move immediately — you already know it is too bad to pick. Cutting off these doomed lines lets you search far fewer positions for the exact same final answer.

UNDER THE HOOD

Alpha-beta keeps two bounds: alpha (the best score the side-to-move is already guaranteed) and beta (the best the opponent will allow). When a move's score meets or exceeds beta, it is a "cutoff" — the rest of that node's moves are irrelevant and we prune. Alpha-beta returns the identical move to plain minimax while visiting roughly the square root of the nodes, given good move ordering.

Move ordering — why trying good moves first is everything

IN PLAIN ENGLISH

Alpha-beta only saves work if you look at strong moves early (that is what triggers the cutoffs). So before searching, we sort the moves to put the likely-best ones first.

UNDER THE HOOD

Ordering priority: the best move remembered from the transposition table, then captures scored by MVV-LVA (Most Valuable Victim minus Least Valuable Attacker — grab the biggest piece with your smallest one), then two "killer moves" (quiet moves that caused cutoffs at the same depth elsewhere), then a "history" table that rewards quiet moves that have been good throughout the search. Good ordering is the single biggest multiplier on alpha-beta's effectiveness.

Iterative deepening — search shallow, then deeper, then deeper

IN PLAIN ENGLISH

Instead of going straight for, say, 10 moves deep, we search 1 deep, then 2, then 3, all the way up. It sounds wasteful to redo the shallow searches, but it is actually faster overall — and it means we always have a good move ready if we run out of time.

UNDER THE HOOD

Each depth's results seed the next: the best move found becomes the first move tried next time, which makes alpha-beta prune dramatically better. It also gives "anytime" behaviour — stop at the deadline and return the best move from the last completed depth. This is how the time budget and the depth presets coexist.

Transposition table — remember positions you've already scored

IN PLAIN ENGLISH

Chess often reaches the same position by different move orders. Rather than re-think a position from scratch, we cache the result the first time and look it up later.

UNDER THE HOOD

A hash table keyed by the Zobrist position key stores depth, score, a bound type (exact / lower-bound / upper-bound) and the best move. On revisiting a position at sufficient depth we can return or tighten bounds immediately. Crucial subtlety: entries are tagged so that scores produced by one evaluator are never reused by the other — switching HCE↔NNUE flushes the table.

Quiescence search — don't stop in the middle of a fight

IN PLAIN ENGLISH

If we stop looking exactly when pieces are mid-trade, we get a nonsense score ("I'm up a queen!" — ignoring that it gets recaptured next move). So at the bottom of the search we keep playing out just the captures until the position is calm, then score it.

UNDER THE HOOD

At depth zero we enter quiescence: search only captures and promotions until a quiet position, using a "stand-pat" lower bound. This cures the horizon effect for tactics and is essential for a shallow fixed depth to evaluate sanely.

The pruning heuristics (extra speed)

- **Null-move pruning** — In plain English: "if I let my opponent move twice in a row and I'm *still* winning, this position is so good I don't need to search it carefully — prune it." Technically: search a reduced-depth null window after giving the opponent a free move; if it still fails high, cut. Skipped in check and in pawn-only endings (where passing can actually be bad — "zugzwang").
- **Late-move reductions (LMR)** — In plain English: moves we try late are probably weak, so we look at them less deeply, and only re-examine fully if one surprises us by being good. Technically: reduce depth for late, quiet, non-check moves; re-search at full depth on a beat-alpha.
- **Aspiration windows** — In plain English: we expect the new score to be near the last one, so we search with a narrow guess-window and only widen it if reality lands outside. Technically: centre a small alpha-beta window on the previous iteration's score; widen on fail-high/low.

The headline knob the player actually sees — the model's **depth** — controls how far this whole machine looks ahead. More depth means more foresight, which means stronger play and more time per move. That is exactly why the AI presets are labelled by depth.

5 How the AI judges a position: evaluation

The search can't play games to the very end — the tree is astronomically large. So at the leaves it needs a number that says "how good is this position for me, right now?" That number is the **evaluation**, measured in **centipawns** (hundredths of a pawn): +100 means "up about a pawn", -300 means "down about a knight". We have two evaluators, and the player picks which one the AI uses.

HCE — the hand-crafted evaluator (the "classical" brain)

IN PLAIN ENGLISH

A human (with help) wrote down the rules of thumb every club player knows — count material, put knights in the centre, keep your king safe, avoid doubled pawns, put rooks on open files — and turned each into points. Add them up and you get a score. No training, no data; just encoded chess wisdom.

UNDER THE HOOD

Material plus positional terms: piece-square tables, king safety (pawn shield, attack counts, open files near the king), pawn structure (doubled/isolated/backward/passed/connected), rook activity, mobility, threats, and more — blended between middlegame and endgame by a "phase" value computed from remaining material. In engine literature this is exactly what is called a "classical" evaluation, as opposed to a neural one — which is why the HCE opponents are named "Classically ...".

NNUE — the learned evaluator (the "neural" brain)

IN PLAIN ENGLISH

Instead of a human writing rules, we showed a small neural network millions of positions each labelled with how good it was, and let it learn its own scoring function. It discovers patterns a human might never write down explicitly.

NNUE stands for "Efficiently Updatable Neural Network" (the letters are deliberately backwards). Section 6 explains how it is built and Section 7 explains the word "efficiently", which is the whole engineering story.

THE DECISION & WHY

Why keep both evaluators? The HCE is a strong, dependable baseline and a yardstick: it lets us measure whether the neural net is actually getting better (play them against each other). Keeping both also makes a nice product feature — two distinct styles of opponent — and the two **never mix**: an NNUE opponent uses only the net, a Classical opponent uses only the HCE. We even made the app refuse to start an NNUE game if the net fails to load, rather than silently falling back to the HCE and lying about which brain is playing.

6 NNUE, properly

What a neural network is, minus the mystique

IN PLAIN ENGLISH

A neural network is just a big mathematical function with millions of adjustable knobs ("weights"). You feed numbers in one end, they get multiplied and added through a few layers, and a number comes out the other end. "Training" means automatically nudging the knobs until the output matches the examples you showed it. There is no magic — it is curve-fitting at scale.

UNDER THE HOOD

Layers of weighted sums separated by a non-linear "activation" function (without the non-linearity, stacked layers would collapse into a single linear function). Training uses gradient descent: measure the error, compute which way to nudge each weight to reduce it, repeat over millions of examples.

The inputs: HalfKP features

IN PLAIN ENGLISH

How do you describe a chess position to a network as a list of numbers? NNUE uses a clever encoding: for each piece on the board, it switches on a feature that says "*my king is on this square AND there is this kind of piece on that square.*" Tying every feature to where your own king is means the network automatically learns king-safety ideas — the same piece is judged differently depending on where your king sits.

UNDER THE HOOD

HalfKP = (King square, Piece square + type), per perspective. There are 64 king squares × 10 non-king piece types (5 of mine, 5 of the opponent's) × 64 squares = **40,960** possible features, of which only ~30 are active in any position. The board is encoded twice — once from each side's perspective — so the network always sees "me vs them".

The shape of the net

40960 inputs (HalfKP features) > 256 (per side, x2 = 512) the "accumulator" > 32 > 32 > 1
two hidden layers score (centipawns)

UNDER THE HOOD

The first layer (the "feature transformer") maps the ~30 active inputs to a 256-number summary per side — the **accumulator**. Both accumulators (512 numbers) feed two 32-wide hidden layers, then a final layer outputs one centipawn score. Activations are a **clipped ReLU** (values squashed into a fixed 0..1 range), which is what makes the later integer math behave.

THE DECISION & WHY

NNUE over a big deep network. A large convolutional net might evaluate a position more accurately, but it would be far too slow to call millions of times inside the search. NNUE is deliberately small and structured so its evaluation can be updated incrementally as moves are made (Section 7) — accuracy traded for the raw call-rate a search needs.

THE DECISION & WHY

HalfKP inputs over simpler piece-square inputs. Feeding raw "piece on square" features is simpler but blind to king safety. Coupling every feature to the king's square is what lets the net learn king safety implicitly — at the cost of a much larger (but extremely sparse) input space.

7 The efficiency tricks (the heart of the project)

A neural network that is called millions of times per move must be brutally cheap to evaluate. Three tricks make that possible. Each is presented as **problem → insight →**

how, and for two of them we found the original code was leaving most of the benefit on the table and fixed it.

Trick 1 — the incrementally-updated accumulator

IN PLAIN ENGLISH

Computing that 256-number summary from scratch for every position is the single most expensive thing the net does. But between one position and the next, almost nothing changes — a move usually just moves one piece. So instead of rebuilding the summary, we **patch only the part that changed**: subtract the piece that left, add the piece that arrived. One or two small updates instead of thirty.

UNDER THE HOOD

Each search node keeps an accumulator on a stack. On make-move we compute the new active-feature list and apply the **symmetric difference** versus the parent (an $O(n)$ merge of two short sorted lists): subtract the columns for features that turned off, add the columns for features that turned on. On unmake we just pop the stack. This is literally what the "U" (Updatable) in NNUE refers to.

The king-move special case (and why HalfKP forces it)

IN PLAIN ENGLISH

There is one move that ruins the shortcut: moving your own king. Because every feature is tied to your king's square, if the king moves, *every single feature changes at once* — there is nothing to reuse. So for king moves we just rebuild that side from scratch; for every other move we take the cheap path.

UNDER THE HOOD

When the side-to-move's king square changes, the entire perspective is recomputed (a full refresh of that side's accumulator). Detecting this is a single square comparison. Every non-king move stays on the incremental path.

THE DECISION & WHY

What we fixed. The original "incremental" update recomputed the full feature list for both sides every node and diffed them with an $O(n^2)$ double loop — so much overhead that it was only $\sim 1.08x$ faster than a full rebuild (i.e. barely incremental at all). We rewrote it as a sorted set-difference with the king-move refresh special-case, taking it to $\sim 2x$ in a scalar build and more under SIMD. It is verified **bit-for-bit identical** to a from-scratch rebuild over long move sequences including captures, castling, promotions and king moves.

Trick 2 — quantization (integers instead of decimals)

IN PLAIN ENGLISH

The net is trained using decimal numbers (floats). Whole-number (integer) math is faster and the numbers take less space. So once, when the net loads, we convert every weight into a small integer and do all the actual game-time math in integers — then scale the answer back to centipawns at the end.

UNDER THE HOOD

Activations are scaled by QA=127 (to fit an 8-bit-ish range) and weights by QW=64. Feature-transformer weights are stored as int16; the dense-layer weights as int8 or int16. The final integer accumulation is divided back down to centipawns. Three on-disk formats exist and the engine auto-detects which it is loading:

Format	What it stores	Size
NNU1	raw float weights (no quantization)	~42 MB
NNU2	int8 weights (the shipped default)	~10.5 MB
NNU3	int16 feature transformer	~21 MB

Quantization is verified **faithful**: the float, int8 and int16 evaluations agree to within rounding, so making it fast did not change how the net plays.

Trick 3 — SIMD (do eight multiplications in one instruction)

IN PLAIN ENGLISH

Modern processors can do the same operation on several numbers at once — like an assembly line that stamps eight parts per pull of the lever instead of one. The net's inner loops are nothing but "multiply these numbers and add them up", which is exactly what this is good at.

UNDER THE HOOD

WebAssembly SIMD operates on 128-bit vectors — eight 16-bit integers per instruction. The accumulator add/subtract uses i16x8 vector ops; the dense layers — the real hotspot — use i32x4_dot_i16x8, one instruction that multiplies eight int16 pairs and sums them. The same code falls back to scalar loops when SIMD is unavailable (the native tournament/benchmark builds), so results are always identical, just slower without SIMD. The browser build is compiled with -msimd128.

THE DECISION & WHY

What we fixed. Originally only the accumulator add/subtract used SIMD; the dense layers — which are the bulk of the per-evaluation cost — ran as plain scalar loops. That was the actual reason the NNUE felt slow. Vectorising the dense dot-products cut the per-evaluation time by roughly 3-4x. Lesson: profile to find the *real* hotspot instead of optimising the part that was already cheap.

The bug behind the bug: the net wasn't loading at all

IN PLAIN ENGLISH

For a while the neural opponents felt suspiciously weak and fast. The real cause wasn't the search or the eval — the net file simply **wasn't loading**, so the engine quietly used the hand-crafted brain instead. The symptom ("weak/odd") pointed nowhere near the cause (a file-format mismatch).

UNDER THE HOOD

The shipped net.nnue was an older layout with int8 dense weights; the loader had since moved to int16 dense weights, so the size check failed and `load_nnue` returned false — silently falling back to HCE. The fix made the loader **back-compatible**: it disambiguates the two layouts by total file length and reads the dense weights at the correct width. We also removed the silent fallback so an unloadable net now refuses to start rather than masquerade as something it isn't.

How we know it all actually works (verification)

None of the above is trustworthy without proof, so we built a benchmark harness (`scripts/nnue_bench`) that: (1) plays a long sequence of moves and asserts the incrementally-updated accumulator is **exactly equal** to a from-scratch rebuild after every move; (2) checks the quantized evaluation tracks the float reference; and (3) times the update path and the raw evaluations per second. A separate native tournament tool plays two models against each other over many games to confirm relative strength. The habit — *change something, then measure that it is both correct and faster* — is the part most worth keeping.

8 The opening book

IN PLAIN ENGLISH

In the opening, the best moves are already known theory — there's no point making the AI think them out. So before searching, the AI checks a book of openings. If the current position is in the book, it instantly plays a known good move (chosen with a bit of randomness so it doesn't always play the same line). The moment the position leaves the book, it goes back to thinking.

UNDER THE HOOD

The book is a standard Polyglot .bin (~380k positions, 16-byte entries of {key, move, weight, learn}). The probe is done in JavaScript before the search is dispatched, for both NNUE and HCE opponents.

The Polyglot key is a different hash on purpose

UNDER THE HOOD

Polyglot books are keyed by a **fixed, standardised** Zobrist hash defined by 781 published constants — not the engine's own internal Zobrist keys. To probe the book we had to hash positions with exactly those 781 numbers. We took them verbatim from the python-chess reference and **verified** our JavaScript hash reproduces python-chess's keys (e.g. the start position hashes to the canonical 0x463b96181691fc9c).

THE DECISION & WHY

Probing the book in JavaScript, not C++. We could have loaded the book into WASM. Doing it in JS meant no engine rebuild, trivial string/UCI handling, and the move just plays through the normal path. The book also encodes castling as "king onto its own rook" (e.g. e1->h1), which we translate to the normal king move. Moves are picked weighted-randomly so openings vary.

One honest limitation: a Polyglot file contains moves but **no opening names**, so book moves are labelled simply "Book move". Showing names like "Sicilian Defence" would need a separate ECO dataset.

9 Training the neural net

IN PLAIN ENGLISH

The network is only as good as what it learned from. Making one is a four-step pipeline: get labelled positions, convert them to the network's input format, train, and export the result into the small file the game loads.

UNDER THE HOOD

(1) **Data** — labelled positions (Lichess games with Stockfish evaluations) as Parquet shards. (2) **Preprocess** — each position becomes its HalfKP features plus a target score, packed into a fast binary `train.bin`. (3) **Train** — PyTorch, gradient descent, with a Colab notebook for free-GPU training. (4) **Export** — quantize the trained weights and write `frontend/models/net.nnue` (NNU2 by default). Since a static host has no "upload", adding a model literally means committing the file.

THE DECISION & WHY

Ship the undertrained net, improve later. Our net plays real chess but is undertrained — its evaluation saturates around +5 pawns (it can't tell a queen from a rook advantage), so at equal depth it is a bit weaker than the hand-crafted eval. Crucially, the **engine and quantization are verified faithful** — the weakness is training (more data, more epochs, a GPU), not code. Because evaluation sits behind an interface, a better net drops straight in with zero engine changes. So we shipped a working pipeline and an honest baseline rather than blocking on a perfect net.

10 The front end (and why it's plain JavaScript)

IN PLAIN ENGLISH

The web page deliberately uses no framework (no React, no Vue) — just hand-written HTML, CSS and JavaScript. The page's only job is to draw the board and pass clicks to the engine; all the heavy lifting is in WASM, so a framework would add weight and start-up cost for no benefit.

THE DECISION & WHY

Vanilla JS over a framework. The top priority was fast load and zero runtime overhead. A framework bundle and virtual-DOM diffing buy nothing when the only dynamic thing is an 8x8 grid and the real compute lives in WebAssembly. So we kept the front end tiny and dependency-free.

A few front-end decisions worth knowing

- **One config object for the opponents.** All six AI presets live in a single `MODELS` object (label, eval type, depth, time budget, description); the dropdown is generated from it, so adding or renaming an opponent is a one-line edit in one place.
- **The honest move log.** After each AI move the log shows the evaluator that *actually* ran and the depth reached (e.g. "via NNUE D10"), computed from the real engine state — not just the model's display name. This is what let us prove NNUE and HCE were genuinely different and not silently falling back.
- **We removed the evaluation bar.** The vertical "who's winning" bar beside the board ate horizontal space that matters on a phone, so we cut it and gave that width back to the board. A small product call: board real estate beats a nicety on mobile.
- **Mobile + polish.** The board sizes itself to the viewport, the UI text isn't accidentally selectable, and a black/gold/white theme replaces the generic default look.

11 Every design choice in one table

Interviewers love "why did you choose X?" Here is every significant decision, the alternative we turned down, and the reason — the whole project's judgement, on one page.

Decision	We chose	Over	Why
Rules + AI language	C++ → WASM	JavaScript / chess.js	Speed for millions of nodes; one engine is the single source of truth
Board representation	Bitboards	Mailbox array	Bitwise ops replace loops — far faster move-gen and counting
Where the AI runs	Client (WASM)	Backend server	No server cost, no latency, scales free, hosts as static files

Decision	We chose	Over	Why
WASM compiler	Zig's clang	Emscripten	Emscripten was network-blocked; Zig gives a standard wasm32 module
Shipping the WASM	base64-embedded	Separate .wasm fetch	Works from file://, no CORS/MIME issues, one fewer request
Search threading	Web Worker	Main thread	UI never freezes while the AI thinks
Evaluation seam	Abstract Evaluator	Hard-wired eval	Lets NNUE and HCE swap in/out and be switched at runtime
Eval network	Small NNUE	Large deep net	Must be callable millions of times/move; supports incremental update
Net inputs	HalfKP	Plain piece-square	King-coupled features let it learn king safety implicitly
Net arithmetic	Quantized int	Float	Integer math is faster and smaller; verified faithful
Accumulator	Incremental + king-refresh	Full rebuild each node	Patch only what changed; ~all the speed of "efficiently updatable"
Hot loops	WASM SIMD	Scalar	8 multiply-adds per instruction on the dense-layer hotspot
HCE vs NNUE	Strictly separate	Silent fallback	Predictable, honest opponents; never lie about which brain plays
Opening moves	Polyglot book	Hardcoded / none	Standard, large, free; instant known-good opening play
Book probing	JavaScript	Inside WASM	No rebuild, easy move handling, plays through the normal path
Front end	Vanilla JS	React/Vue	Fast load, zero runtime overhead; compute lives in WASM
Eval bar (mobile)	Removed	Kept	Board real estate beats the nicety on a phone

12 War stories: the bugs and what they taught us

"The NNUE is weak and fast" — it wasn't even running

The neural opponents felt off. The instinct is to blame the net or the search. The real cause was a **file-format mismatch** that made the net fail to load, so the engine silently used the hand-crafted eval. **Lesson:** a symptom can point miles from its cause; verify your assumption ("is the net even loaded?") before optimising the thing you assume is slow.

"NNUE and HCE play the same move" — coincidence, not a bug

After the fix, the two brains sometimes picked the same move and it looked like the toggle was broken. In fact, on obvious positions two good evaluators often *agree on the move* while still producing different scores and searching different numbers of nodes. **Lesson:**

same output isn't proof of same process — we added a log that reports the evaluator that truly ran to settle it for good.

"It's faster on one server than another" — it was the browser cache

The same build appeared to run at very different speeds depending on which local web server served it. A static file server cannot change how fast WASM executes — the difference was the **browser cache** serving a stale, older build under one server's address.

Lesson: when two "identical" things behave differently, suspect what is actually different (cached bytes, headers), not the thing you are looking at.

Removing the silent fallback on purpose

Once NNUE loaded correctly, we deliberately made an NNUE opponent **refuse to start** if the net can't load, instead of quietly playing HCE. **Lesson:** a program that silently does something other than what its label says is worse than one that honestly errors — especially when you are trying to reason about it.

13 Mock interview: questions you can now answer

Q Give me a one-minute overview of the project.

"It's a browser chess game where the rules and the AI are written in C++ and compiled to WebAssembly, so everything runs client-side at near-native speed with no server. The engine is the single source of truth for legality. The AI does an alpha-beta search with the usual enhancements, and it can evaluate positions two ways: a hand-crafted 'classical' evaluator, or an NNUE neural network I trained and made fast with quantization, an incremental accumulator, and SIMD. There's a Polyglot opening book, and the search runs in a Web Worker so the UI never freezes."

Q What is NNUE and why is it 'efficient'?

"It's a small neural network for position evaluation. 'Efficiently updatable' means that when a move is made, instead of recomputing the network's first-layer summary from scratch, you patch only the few inputs that changed — add the piece that arrived, subtract the one that left. With the HalfKP input encoding the one exception is moving your own king, which changes every feature, so that case does a full refresh. Combined with integer quantization and SIMD, that makes it cheap enough to call millions of times per move."

Q Explain alpha-beta pruning to someone non-technical.

"You look ahead assuming both players play their best. While checking a move, if you find the opponent has a reply so strong that you'd never choose this move anyway, you stop looking at it immediately — you already know enough to reject it. Skipping those settled lines lets you search far fewer positions for the exact same decision. The catch is you must try promising moves first, which is why move ordering matters so much."

Q What is SIMD and where did you use it?

"SIMD is one CPU instruction doing the same math on several numbers at once — eight 16-bit integers per instruction in WebAssembly. The neural net's inner loops are just multiply-and-add, which is exactly what SIMD accelerates. I used it for the accumulator updates and, more importantly, the dense layers, which were the real hotspot. Vectorising the dense dot-products cut evaluation time by 3-4x."

Q What was the hardest or most instructive bug?

"The neural opponents seemed weak and fast, and I assumed the net or the search was the problem. The actual cause was a file-format mismatch — the net wasn't loading at all, so it silently fell back to the hand-crafted eval. The lesson was to verify the assumption ('is the net even loaded?') instead of optimising the thing I assumed was slow. I fixed the loader to be back-compatible and removed the silent fallback so it can never lie about which evaluator is running."

Q Why WebAssembly instead of just JavaScript, or a server?

"A chess search touches millions of positions per move; the integer-heavy inner loops run much faster as compiled C++ in WASM than in JS, while JS still handles the DOM. And doing it client-side instead of on a server means no latency per move, no hosting cost, free scaling, and it deploys as static files."

Q How did you verify the optimisations didn't break correctness?

"A benchmark harness asserts the incrementally-updated accumulator is bit-for-bit identical to a from-scratch rebuild after every move — captures, castling, promotions, king moves — and that the quantized eval tracks the float reference. A separate tournament tool plays models against each other over many games. The discipline is: every change is checked for correctness and speed, not just speed."

Q Why is your NNUE weaker than the hand-crafted eval right now?

"It's undertrained — its scores saturate around +5 pawns, so it can't distinguish a queen advantage from a rook advantage, which costs it at equal depth. The engine and quantization are verified faithful, so the gap is training data and compute, not code. Because evaluation sits behind an interface, a better-trained net drops in with no engine changes."

Q What does 'depth' mean and why does more of it play better?

"Depth is how many plies — half-moves — the search looks ahead. More depth means it sees further consequences before committing, so it avoids more traps and finds more tactics, at the cost of more time per move. That's why the difficulty presets are just different depths."

Q What would you improve next?

"Train a stronger net (more data, more epochs, GPU); add opening names via an ECO dataset; and possibly a deeper incremental update that derives changed features directly from the move rather than recomputing the feature list. None of these require engine changes thanks to the interface seam."

14 Glossary: every piece of jargon, defined

Term	Plain-English meaning
WebAssembly (WASM)	A compact format that runs compiled code (our C++) in the browser at near-native speed.
SIMD	"Single Instruction, Multiple Data" — one CPU instruction doing the same math on several numbers at once.
Bitboard	A 64-bit number used as the board: one bit per square, set where a given piece is.
Mailbox	The alternative board layout: a 64-entry array naming the piece on each square. Simpler, slower.
NNUE	"Efficiently Updatable Neural Network" — a small neural net for position evaluation, cheap to update move-to-move.
HalfKP	NNUE's input encoding: features of the form (your king's square, a piece + its square).
Accumulator	The 256-number per-side summary the net keeps and updates incrementally; the first layer's output.
Quantization	Converting the net's decimal weights to small integers so the math is faster and the file smaller.
Centipawn	The unit of evaluation: 1/100 of a pawn. +100 = up about a pawn.
Evaluation / eval	A number scoring how good a position is for the side to move.
HCE	Hand-Crafted Evaluation — human-written chess heuristics turned into a score (the "classical" eval).
Minimax / negamax	Choosing your move assuming the opponent always replies with their best; negamax is the compact form.
Alpha-beta	A way to skip lines that can't change the decision, giving the same answer as minimax far cheaper.
Ply	One half-move (one player's single move). "Depth 10" = ten plies of look-ahead.
Iterative deepening	Searching depth 1, then 2, then 3... — faster overall and always has a move ready.
Transposition table	A cache of already-scored positions, keyed by a Zobrist hash.
Zobrist hash	A 64-bit fingerprint of a position, built by XOR-ing random per-feature keys.
Quiescence search	Extra search of only captures at the leaves so we don't stop mid-trade with a wrong score.
Null-move pruning	"If I pass and I'm still winning, prune this line." A fast way to cut clearly-winning branches.
LMR	Late-Move Reductions — search likely-bad late moves shallower, re-search fully only if one surprises us.

Term	Plain-English meaning
Aspiration window	Searching with a narrow guess-window around the expected score; widen only if it's wrong.
MVV-LVA	Most Valuable Victim, Least Valuable Attacker — a rule for ordering captures first.
Killer / history	Move-ordering memory: quiet moves that caused cutoffs before are tried earlier.
Clipped ReLU	The net's activation function: squashes each value into a fixed 0..1 range.
Polyglot book	A standard opening-book file format mapping a position hash to weighted moves.
Web Worker	A background browser thread; we run the search there so the UI thread stays free.
Perft	A move-generator correctness test: count all positions to depth N and compare to known totals.
FEN	A short text string that fully describes a chess position.

That's the whole project. You set the direction, chose between the options at every fork, and found the symptoms that led to the real fixes. If you can give the plain-English answer and then the under-the-hood one for each topic here, you can hold your own on this project in any room.